

CI/CD, Testing, and Full-Stack Systems

What is Continuous Integration/Continuous Delivery (CI/CD)?

Continuous Integration means:

- automatically checking code on every push
- running tests before merge
- enforcing code quality consistently
- catching regressions early

CI answers: 'Is this change safe to integrate?'

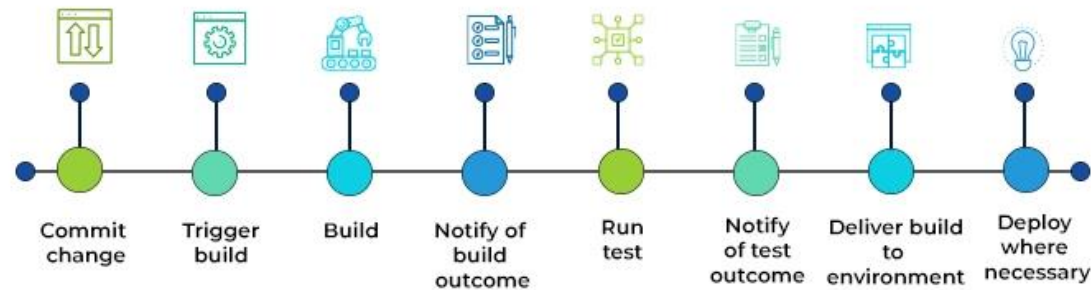
Continuous Delivery / Deployment means:

- packaging software reproducibly
- automating release steps
- reducing manual deployment errors
- making changes easier to ship

CD answers: 'How does passing code become running software?'



CI/CD PIPELINE



Example: Full-Stack App

``examples/fullstack-app/``

- FastAPI backend
- Streamlit frontend
- backend tests
- compose setup
- CI workflow
- Mermaid diagrams

One repository is enough to show testing, automation, deployment structure, and architecture and to discuss full engineering workflow.

This repo already includes:

- an API contract
- a user interface
- automated tests
- containerized local setup
- CI checks
- architecture documentation

FastAPI Backend Structure

```
app = FastAPI(title="fullstack-backend")
```

```
class PredictRequest(BaseModel):
```

```
    sepal_length: float
```

```
    sepal_width: float
```

```
    petal_length: float
```

```
    petal_width: float
```

```
CACHE: dict[tuple[float, float, float, float], str] = {}
```

```
PREDICTION_LOG: list[dict[str, float | str | bool]] = []
```

```
@app.get("/health")
```

```
def health():
```

```
    return {"status": "ok"}
```

```
@app.get("/history")
```

```
def history():
```

```
    return {"requests": PREDICTION_LOG}
```

```
@app.post("/predict")
```

```
def predict(request: PredictRequest):
```

```
    ...
```

Prediction Logic with Cache and History

```
@app.post("/predict")
def predict(request: PredictRequest):
    features = (
        request.sepal_length,
        request.sepal_width,
        request.petal_length,
        request.petal_width,
    )
    if features in CACHE:
        prediction = CACHE[features]
        cached = True
    else:
        predicted_class = model.predict([list(features)])[0]
        prediction = target_names[int(predicted_class)]
        CACHE[features] = prediction
        cached = False

    record = {**request.model_dump(), "prediction": prediction, "cached": cached}
    PREDICTION_LOG.append(record)
    return record
```

Backend Tests

```
def test_health_endpoint():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_predict_endpoint_returns_real_iris_prediction_and_cached_flag():
    response = client.post(
        "/predict",
        json={"sepal_length": 5.1, "sepal_width": 3.5, "petal_length": 1.4, "petal_width": 0.2},
    )
    assert response.status_code == 200
    body = response.json()
    assert body["prediction"] in {"setosa", "versicolor", "virginica"}
    assert body["cached"] is False

def test_predict_endpoint_uses_cache_on_repeat_request():
    ...
```

CI for the Full-Stack Repository

jobs:

 backend-tests:

 runs-on: ubuntu-latest

 defaults:

 run:

 working-directory: week-9/examples/fullstack-app/backend

 steps:

- uses: actions/checkout@v4
- uses: actions/setup-python@v5
- uses: astral-sh/setup-uv@v6
- run: uv sync
- run: uv run pytest -q

 frontend-smoke-test:

 runs-on: ubuntu-latest

 defaults:

 run:

 working-directory: week-9/examples/fullstack-app/frontend

 steps:

- uses: actions/checkout@v4
- uses: actions/setup-python@v5
- uses: astral-sh/setup-uv@v6
- run: uv sync
- run: uv run python -m py_compile app.py

How to Read a GitHub Actions Workflow

Every workflow answers four questions:

1. When should it run?
2. What runner should execute it?
3. What environment must be prepared?
4. What commands define pass/fail?

If those four questions are clear, the YAML is usually understandable.

On push or pull request, GitHub Actions:

- checks out the repository
- sets up Python 3.11
- installs `uv`
- runs `uv sync` in the backend
- runs `uv run pytest -q`
- runs `uv sync` in the frontend
- runs `uv run python -m py\_compile app.py`

No secrets are required for this CI workflow.



Example File vs Runtime Workflow Location

For class, we keep the readable example here:

- ``week-9/examples/fullstack-app/ci.yml``

For GitHub Actions to actually run it, the workflow must exist here:

- ``github/workflows/week9-fullstack-ci.yml``

Same idea, different purpose:

- example next to the app
- executable workflow in ``github/workflows/``

Triggers, Jobs, and Steps

Core GitHub Actions concepts:

- `push` and `pull\_request` define when automation runs
- jobs are separate runner environments
- steps are individual actions or commands inside a job
- `needs` can enforce ordering between jobs

Quality Checks in CI

CI can enforce more than tests:

- linting with Ruff
- formatting with Black
- type checks if desired
- security or dependency scanning later

Tests check behavior; the rest keeps the repo consistent and reviewable.

Typical Deployment Workflow Shape

name: Deploy

on:

push:

branches: [main]

jobs:

deploy:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: google-github-actions/auth@v1

with:

credentials_json: \${{ secrets.GCP_SA_KEY }}

- run: |

gcloud run deploy my-app \

--source . \

--region us-central1

Containers in the Delivery Path

Containers define:

- base operating system image
- dependency installation
- copied source code
- startup command

That makes local testing and deployment more reproducible.

```
FROM python:3.11-slim
```

```
WORKDIR /app
```

```
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
```

```
COPY pyproject.toml uv.lock .
```

```
RUN uv sync --no-dev
```

```
COPY main.py .
```

```
CMD ["uv", "run", "python", "main.py"]
```

Interactive System Diagram

The image shows a code editor on the left and a diagram on the right, illustrating the architecture of a web application.

Code Editor (Left):

```
graph LR
  User[User in browser] --> Frontend[Streamlit frontend]
  Frontend -->|POST /predict| Backend[FastAPI backend]
  Backend -->|GET /history| Frontend
  Backend --> Cache[(In-memory cache)]
  Backend --> History[(Request history)]
  Backend -->|JSON response| Frontend
  Frontend -->|updated result + history view| User
```

Diagram (Right):

The diagram illustrates the data flow between the components:

- User in browser** sends a request to the **Streamlit frontend**.
- Streamlit frontend** sends a **POST /predict** request to the **FastAPI backend**.
- FastAPI backend** sends a **JSON response** back to the **Streamlit frontend**.
- FastAPI backend** interacts with two databases: **In-memory cache** and **Request history**.
- FastAPI backend** sends a **GET /history** request back to the **Streamlit frontend**.
- Streamlit frontend** sends an **updated result + history view** back to the **User in browser**.

Development and Delivery Loop

Code

Auto-Update Docs

```
1 graph LR
2   Dev[Developer] --> GitHub[GitHub repository]
3   GitHub --> CI[GitHub Actions CI]
4   CI -->|tests + smoke checks pass| Image[Build artifacts / containers]
5   Image --> Deploy[Deployment target]
6   Deploy --> User[User]
7   User --> Frontend[Streamlit frontend]
8   Frontend --> Backend[FastAPI backend]
9   Backend --> Frontend
10  Frontend --> User
11  CI -. feedback on failures .-> Dev
12 Quick edit (⌘⌘)
```



default

^

GET

/health

Health

▼

GET

/history

History

▼

POST

/predict

Predict

📋

▼

Schemas

^

HTTPValidationError >

Expand all

object

PredictRequest >

Expand all

object

ValidationError >

Expand all

object

Backend State

The backend keeps two kinds of in-memory state:

- `CACHE` for repeated feature tuples
- `PREDICTION\_LOG` for request history

That allows the same API to support:

- cached responses
- history inspection
- stateful test cases

Backend Behaviors

The backend demonstrates:

- request validation
- deterministic model inference
- cache hit vs cache miss behavior
- stateful API responses
- stable response contracts for the frontend

Streamlit Frontend Structure

```
BACKEND_URL = os.getenv("BACKEND_URL", "http://backend:8000")
```

```
st.title("Full-Stack Iris Classifier Demo")
sepal_length = st.number_input("Sepal length (cm)", value=5.1)
sepal_width = st.number_input("Sepal width (cm)", value=3.5)
petal_length = st.number_input("Petal length (cm)", value=1.4)
petal_width = st.number_input("Petal width (cm)", value=0.2)
```

```
if st.button("Predict species"):
    response = requests.post(
        f"{BACKEND_URL}/predict",
        json={
            "sepal_length": sepal_length,
            "sepal_width": sepal_width,
            "petal_length": petal_length,
            "petal_width": petal_width,
        },
        timeout=10,
    )
```

Full-Stack Iris Classifier Demo

Enter flower measurements and send them to the FastAPI backend for prediction.

Sepal length (cm)

5.10

- +

Sepal width (cm)

3.50

- +

Petal length (cm)

1.40

- +

Petal width (cm)

0.20

- +

Predict species

Load history

Frontend History Request

```
if st.button("Load history"):
    response = requests.get(f"{BACKEND_URL}/history", timeout=10)
    if response.ok:
        st.json(response.json())
    else:
        st.error("Could not load history")
```

Frontend-Backend Contract

The frontend depends on specific backend behavior:

- `/predict`` accepts four numeric Iris features
- `/predict`` returns ``prediction`` and ``cached``
- `/history`` returns structured JSON under ``requests``

If those contracts drift, the frontend can break even if the backend still runs.

Environment Variables Connect the Layers

The frontend reads:

```
`BACKEND_URL = os.getenv("BACKEND_URL", "http://backend:8000")`
```

That means the same code can run:

- inside compose using the service name
- locally against `localhost`
- in cloud environments with a different URL

Configuration changes without rewriting code.

Compose for Multi-Service Development

Compose gives one command to launch multiple services together.

That helps with:

- reproducible local development
- easier demos
- testing service-to-service communication
- fewer environment-specific surprises

Both images install dependencies with uv during build.

```
services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"

  frontend:
    build: ./frontend
    ports:
      - "8501:8501"
    environment:
      - BACKEND_URL=http://backend:8000
    depends_on:
      - backend
```

Sequence Diagram for User Interaction

Personal files / User Authentication



Export

Share

10 Credits

Start my trial

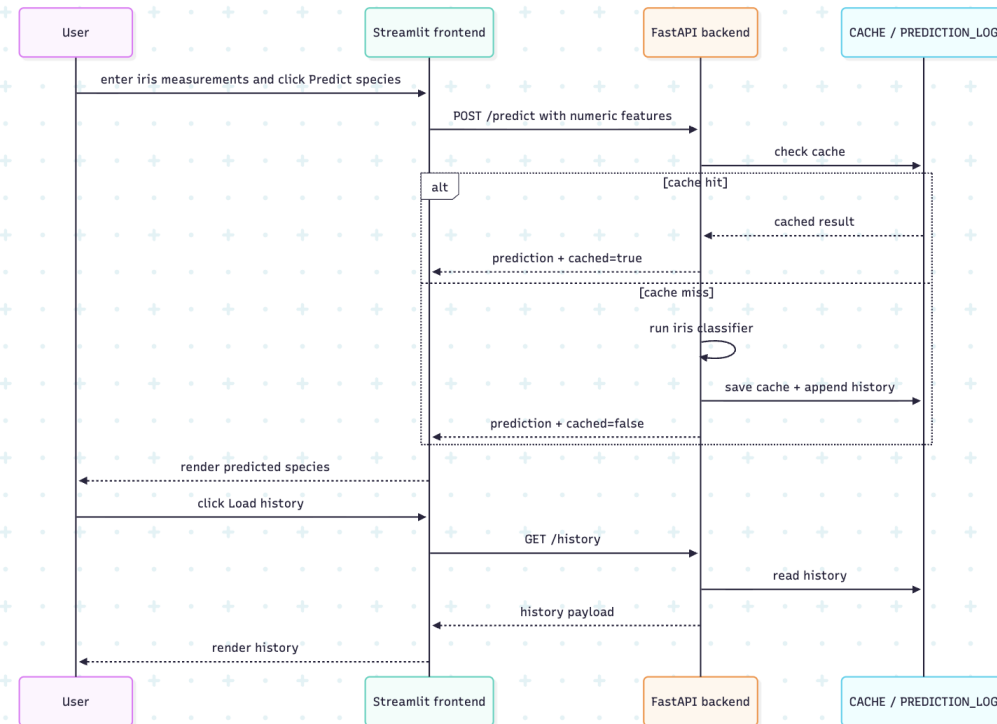
Code

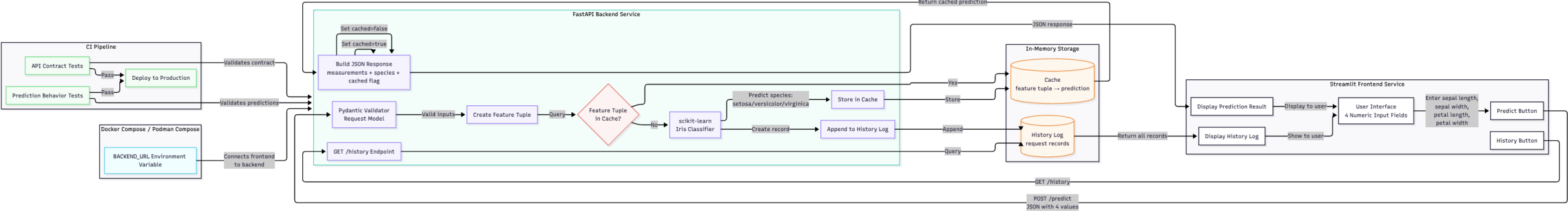
Auto-Update

Docs



```
1 sequenceDiagram
2   participant U as User
3   participant F as Streamlit frontend
4   participant B as FastAPI backend
5   participant C as CACHE / PREDICTION_LOG
6
7   U->>F: enter iris measurements and click Predict species
8   F->>B: POST /predict with numeric features
9   B->>C: check cache
10  alt cache hit
11    C-->>B: cached result
12    B-->>F: prediction + cached=true
13  else cache miss
14    B->>B: run iris classifier
15    B->>C: save cache + append history
16    B-->>F: prediction + cached=false
17  end
18  F-->>U: render predicted species
19  U->>F: click Load history
20  F->>B: GET /history
21  B->>C: read history
22  B-->>F: history payload
23  F-->>U: render history
```





Architecture Best Practices

Start simple, add complexity as needed
Separate concerns (frontend, API, data)
Use environment variables for config
Implement health checks
Log important events
Handle errors gracefully
Document your architecture
Test at every level

Most projects should begin with the simplest useful architecture.

A good progression is:

- one app if possible
- split frontend and backend when needed
- add database, cache, queues, or agents only when there is a real reason

Test Cases in the Repo

The backend tests check:

- service health
- response shape from `/predict`
- cache behavior on repeated requests

These tests exercise the public API rather than internal helper functions.

The tests catch issues such as:

- broken routes
- missing response fields
- cache regressions
- backend changes that would break the frontend

Workflow Jobs

The CI workflow runs separate jobs for:

- backend tests
- frontend smoke checks

Each job has its own:

- working directory
- dependencies
- validation commands

Why the Frontend Uses a Smoke Test

The frontend currently uses a lightweight check instead of a full UI test suite.

That still verifies:

- dependencies install
- code imports successfully
- syntax is valid

As the UI grows, the test strategy can grow too.

Health Checks

Health endpoints answer a key operational question:
'Is the service up enough to respond?'

Examples:

- `/health`` for simple liveness
- `/ready`` when dependencies must also be checked

Cloud platforms and humans both rely on these signals.

CI/CD Best Practices

- Run tests on every commit
- Keep pipelines fast
- Fail fast on errors
- Use caching effectively
- Separate build and deploy stages
- Use environment-specific configs
- Manage secrets properly
- Verify deployments after release

A Practical Engineering Loop

A healthy project workflow looks like:

1. change code
2. test locally
3. commit and push
4. CI validates the repo
5. merge reviewed code
6. deploy through a scripted path
7. verify health and logs

Course Arc: Weeks 1–9

Week 1 — Dev environment: Git, VSCode, AI tools

Week 2 — Command line: Bash, Zsh, Grep, REGEX

Week 3 — Collab + containers: GitHub, Podman, Docker, uv

Week 4 — Data acquisition: APIs, scraping, BeautifulSoup, Selenium

Week 5 — LLM systems: RAG, vector DBs, LangChain, OpenRouter

Week 6 — Interactive apps: Streamlit, deployment, CloudRun, DockerHub

Week 7 — APIs: FastAPI, Flask, OpenAPI

Week 8 — Agents: tool use, MCP clients/servers, FastMCP

Week 9 — Production workflow: CI/CD, Github Actions, Mermaid, Markdown



Some Final General Advice

1. Practice Curiosity Driven Data Science
2. Do public work
3. Build (y)our community

Curiosity Drive Data Science

1. Don't learn every tool but general frameworks and how to solve problems
2. Ask questions and dig into things you find interesting
3. Story telling and insight generation with iterative feedback loops

Do Public Work

1. Speak at events
2. Contribute to open source or a feature to a GitHub repo
3. Build eminence through blogs, other public projects, your own website



Get Involved



<https://www.dataconla.com>



<https://2026.pybeach.org>



**Southern California
Generative AI**

1,490 members



LA Machine Learning

2,913 members



**R Meetup - Real Data
Science USA (formerl...**

2,288 members



**Python Data
Science LA**

2,186 members



SoCal Python

3,042 members



<http://datafest.stat.ucla.edu/>

Done